ĐẠI HỌC QUỐC GIA TPHCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

MÔN HỌC TƯ DUY TÍNH TOÁN

Trợ lý "Du Lịch Hộp Mù" (Blind Box Travelling)

Đề tài: Đồ án Môn Học

Môn học: Tư Duy Tính Toán

Sinh viên thực hiện:

Huỳnh Tuấn Kiệt (24120356)

Lê Đình Huy (24120324)

Đinh Tiến Phát (24120405)

Vương Đắc Gia Khiêm (24120342)

Trịnh Văn Hợp (24120318)

Trịnh Tuấn Đạt (24120285)

Hồ Văn Lực (24120374)

Giáo viên hướng dẫn:

GS. TS. Nguyễn Văn Hướng Dẫn

Mục lục

I Phân tích & Xác định bài toán **1**

I.I Bối cảnh thực tế 1

I.II Xác định Input / Output 1

II Phân rã (Decomposition) & Thiết kế Module **1**

III Các Hàm và Thuật toán trong Hệ Thống **2**

III.I Các Thuật toán tại Module 1 & Module 8 (Data Pipeline & Smart Search) 2

III.II Luồng Random & Gợi ý (Hidden Gem) 3

III.II.1 Hàm tính khoảng cách Haversine 3

III.II.2 Thuật toán tính điểm Hộp mù (Scoring Algorithm) 3

IV Các Công Cụ Sử Dụng Trong Module **4**

V Tiến trình Thực hiện (Theo phân rã Module) **4**

Danh sách bảng

Danh sách hình vẽ

I Phân tích & Xác định bài toán

I.I Bối cảnh thực tế

Giới trẻ hiện nay thường gặp tình trạng "quá tải sự lựa chọn" khi quyết định địa điểm ăn uống, giải trí (hội chứng FOMO và mệt mỏi khi quyết định). Xu hướng "Hộp mù" (Blind box) đang thịnh hành vì mang lại cảm giác bất ngờ và thú vị. Việc kết hợp bản đồ số, AI và trải nghiệm "hộp mù" giải quyết trực tiếp nhu cầu tìm kiếm sự mới mẻ một cách tiện lợi.

I.II Xác định Input / Output

Bài toán được chuyển từ yêu cầu mơ hồ sang các tham số có thể xử lý rõ ràng:

- **Input:**
  - Tọa độ GPS hiện tại của Client: $(Lat_{current}, Lng_{current})$.
  - Bán kính di chuyển tối đa: R (km).
  - Thể loại yêu thích: $Category$ (VD: Cafe, Ăn vặt).
  - Dữ liệu nhóm (Group mode): Danh sách sở thích của n thành viên.
- **Output:**
  - Tọa độ điểm đến duy nhất (bị ẩn tên): $(Lat_{target}, Lng_{target})$.
  - Đoạn văn 30 từ do AI gợi ý về không gian (gợi sự tò mò).
  - Danh sách vật dụng cần mang (dựa trên thời tiết thực tế).
  - Quỹ tiền dự kiến phải chia đều cho nhóm.

II Phân rã (Decomposition) & Thiết kế Module

Hệ thống được thiết kế theo kiến trúc Microservices/Serverless và được phân rã thành các module logic chạy độc lập nhưng liên kết chặt chẽ với nhau. Dựa trên nghiệp vụ, hệ thống bao gồm 8 Module chính:

1. **Thu thập (Module 1):** Cào dữ liệu offline bằng Python $\rightarrow$ Làm sạch và chuẩn hóa $\rightarrow$ Tải lên Supabase (Data Warehouse).

2. **Input (Module 2 & 6):** Sử dụng Geolocation.getCurrentPosition API lấy tọa độ; Ứng dụng Supabase Realtime để gom nhóm người dùng thành viên và tính trung bình sở thích.

3. **Lọc & Xếp hạng (Module 3):** Hoạt động dựa trên thuật toán "Hidden Gem", query trực tiếp vào Database, lọc theo bán kính yêu cầu và tính điểm xếp hạng để tìm ra điểm duy nhất.

4. **Gia tăng giá trị trải nghiệm (Module 4 & 5):** Hệ thống kết hợp thông tin dự báo thời tiết từ OpenWeatherMap cùng với Gemini API (LLM) để sinh ra Content mồi, tạo sự tò mò.

5. **Hiển thị & Điều hướng (Module 7):** Tích hợp thư viện Mapbox API để dẫn đường trên bản đồ tương tác, dùng Canvas-confetti tạo hiệu ứng chúc mừng (pháo hoa) khi khoảng cách $< 200m$.

6. **Tìm kiếm thông minh - Smart Search (Module 8):** Đánh giá và lọc các category cũng như keywords dựa trên Tagging thông minh từ AI, mang lại khả năng phân loại và truy vấn dữ liệu linh hoạt.

III Các Hàm và Thuật toán trong Hệ Thống

III.I Các Thuật toán tại Module 1 & Module 8 (Data Pipeline & Smart Search)

- **Exponential Backoff & Jitter:** Thuật toán quản lý tải khi gặp lỗi từ các API nhà cung cấp. Thời gian chờ tăng theo hàm mũ và cộng thêm yếu tố ngẫu nhiên nhằm tránh tình trạng gửi request đồng loạt khi server API mở lại. Dùng trong file `enricher.js`.

- **Token Bucket (Rate Limiter):** Giới hạn số lượng request theo đơn vị thời gian cho các API Client để không vi phạm chính sách của Google (Gemini API)/Groq.

- **JSON Parse/Repair:** Xử lý sâu vào Response của AI. Dùng cấu trúc vòng lặp Match để tìm đối tượng JSON hợp lệ bị đưa lẫn vào markdown; nếu hỏng sẽ quay lui ép AI sửa format.

III.II Luồng Random & Gợi ý (Hidden Gem)

III.II.1 Hàm tính khoảng cách Haversine

Sử dụng phương pháp Haversine để tính khoảng cách đường chim bay giữa hai điểm trên bề mặt hình cầu dựa vào vĩ độ và kinh độ.

$$a = \sin^2\left(\frac{\Delta Lat}{2}\right) + \cos(Lat_1)\cdot\cos(Lat_2)\cdot\sin^2\left(\frac{\Delta Lng}{2}\right) \tag{1}$$

$$d = R\cdot 2\cdot \text{atan2}(\sqrt{a}, \sqrt{1-a}) \tag{2}$$

Trong đó $R = 6371$ km là bán kính trung bình của Trái Đất.

III.II.2 Thuật toán tính điểm Hộp mù (Scoring Algorithm)

Thuật toán được thiết kế để đưa ra điểm $TotalScore$ cân bằng giữa chất lượng (Quality), độ bí ẩn (Anonymity) và tính bất ngờ (Random).

- **Quality Score (S_q) theo Bayesian Average:** Nhằm khắc phục trường hợp các địa điểm có ít đánh giá nhưng đạt 5 sao cao đột biến. Điểm số được tinh chỉnh kéo về hệ số trung bình của khu vực (m) kết hợp tham số tự tin (C).

$$S_q = \left[\frac{\text{reviews_count}\times\text{rating} + C\times m}{\text{reviews_count} + C}\right] \div 5.0 \tag{3}$$

- **Anonymity Score (S_a):** Điểm địa điểm phải đáp ứng lượng review cận dưới (ngưỡng ≥ 10) để đảm bảo điểm đến hiện hữu. Tuy nhiên, nếu có quá ít lượt đánh giá so với max reviews vùng thì điểm ẩn dật sẽ càng cao để kích thích khám phá.

$$S_a = 1 - \frac{\log_{10}(\text{reviews_count}+1)}{\log_{10}(\text{max_reviews_in_area}+1)} \tag{4}$$

- **Random Score (S_r):** Điểm ngẫu nhiên tạo độ nhiễu an toàn. Dùng `Math.random()` cấu thành.

Công thức tổng hợp Scoring:

$$\text{TotalScore} = (S_q\times 0.45) + (S_a\times 0.30) + (S_r\times 0.25) \tag{5}$$

Điểm có Total Score cao nhất trong kết quả filter sẽ được trả về cho người dùng.

IV Các Công Cụ Sử Dụng Trong Module

- **Python & Selenium WebDriver:** Công cụ tự động hóa trong module Crawler để giả lập thao tác của người dùng lướt bản đồ số và thu thập dữ liệu DOM động từ client.
- **Node.js & Các SDK (`@google/generative-ai`, `groq-sdk`):** Sử dụng trong module Enricher để đẩy dữ liệu phân loại bất đồng bộ siêu tốc.
- **Các mô hình ngôn ngữ lớn (LLMs):** `gemini-2.5-flash` (Google), `llama-3.3-70b-versatile` (Groq), và các public models qua **OpenRouter**. Chúng dùng để cào nhãn thực thể (Analyst) và review kiểm soát nội dung (Supervisor).
- **Next.js (React Framework):** Framework gốc xử lý Frontend UX/UI, Routing và Backend Serverless cho các truy xuất API.
- **Supabase (PostgreSQL Database):** Nơi lưu trữ đám mây cung cấp các truy vấn bảng phức tạp và Row Level Security bảo mật dữ liệu.
- **Phụ trợ API: Mapbox** (hiển thị, marker bản đồ động) và **OpenWeatherMap API** (bản tin thời tiết thời gian thực phục vụ content AI của dự án).
- **TailwindCSS & Framer Motion:** Xây dựng hệ thống UI tối giản với hiệu ứng mượt mà.

V Tiến trình Thực hiện (Theo phân rã Module)

Dựa trên kiến trúc 8 module đã đề ra, tiến độ thực tế được ghi nhận đến hiện tại như sau:

- **Module 1 (Thu thập - Hoàn thành 100%):** Đã viết xong mã nguồn `crawl_data.py` tự động lấy thông tin từ Google Maps bằng Selenium. Làm giàu, gán nhãn tags thành công `data_enriched.json` với AI (`enricher.js`) và đẩy đủ dữ liệu mẫu lên kho chứa hệ thống Supabase.
- **Module 2 & 6 (Input & Group Mode - Đang phát triển):** Đã lấy thành công `lat, lng` của người dùng qua HTML5 Geolocation (Module 2). Về tính năng Gom nhóm và tính trung bình sở thích qua Supabase Realtime (Module 6), hiện đang hoàn tất thiết kế kiến trúc và chuẩn bị code realtime.

- **Module 3 (Lọc & Xếp hạng - Hoàn thành 100%):** Hoàn thiện hàm API `/api/blind-box`. Trích xuất thành công dữ liệu từ DB, loại bỏ địa điểm ngoài bán kính thực tế bằng công thức Haversine; tính điểm và trả về `TotalScore` chính xác nhờ áp dụng hệ số đánh giá tĩnh Bayesian Average kết hợp Logarit.

- **Module 4 & 5 (Gia tăng giá trị trải nghiệm - Hoàn thành 100%):** Đã thiết lập xong Engine Prompt tại `/api/ai-experience`. Fetch đồng thời dữ liệu OpenWeatherMap phục vụ logic "thời tiết hiện tại" và gọi LLM thông qua Groq API trả về các đoạn phân tích tò mò mập mờ hoàn hảo.

- **Module 7 (Hiển thị & Điều hướng - Đang hoàn thiện):** Đã nhúng xong thư viện Mapbox trên trang UI Next.js, tạo Marker thành công lên bản đồ ứng dụng. Tính năng dẫn đường chi tiết và phát động Canvas-confetti (khi tới gần $< 200m$) đang được tối ưu kiểm thử.

- **Module 8 (Smart Search - Chờ ráp UI):** Thuật toán gắn nhãn tìm kiếm (Tagging/Categorizing) trong dữ liệu đã phủ đủ và chuẩn hóa. Cấu trúc Back-end hỗ trợ lọc thông minh đã có, phần UI box Smart Search trên ứng dụng Frontend đang được chắp nối để chạy hoàn chỉnh.